Evaluating Efficiency, Effectiveness and Satisfaction of AWS and Azure from the Perspective of Cloud Beginners

Gabriel Costa Silva*
Universidade Tecnológica Federal do Paraná
Cornélio Procópio, Paraná, Brazil
gabrielcosta@utfpr.edu.br

Reginaldo Ré
Universidade Tecnológica Federal do Paraná
Campo Mourão, Paraná, Brazil
reginaldo@utfpr.edu.br

Marco Aurélio Graciotto Silva
Universidade Tecnológica Federal do Paraná
Campo Mourão, Paraná, Brazil
magsilva@utfpr.edu.br

ABSTRACT

Quality has long been regarded as an important driver of cloud adoption. In particular, quality in use (QiU) of cloud platforms may drive cloud beginners to the cloud platform that offers the best cloud experience. Cloud beginners are critical to the cloud market because they currently represent nearly a third of cloud users. We carried out three experiments to measure the QiU (*dependent variable*) of public cloud platforms (*independent variable*) regarding efficiency, effectiveness and satisfaction. AWS EC2 and Azure Virtual Machines are the two cloud services used as representative proxies to evaluate cloud platforms (*treatments*). Eleven undergraduate students with limited cloud knowledge (*participants*) manually created 152 VMs (*task*) using the web interface of cloud platforms (*instrument*) following seven different configurations (*trials*) for each cloud platform. Whereas AWS performed significantly better than Azure for efficiency (p-value not exceeding 0.001, A-statistic = 0.68), we could not find a significant difference between platforms for effectiveness (p-value exceeding 0.05) - although the effect size was found relevant (odds ratio = 0.41). Regarding satisfaction, most of our participants perceived the AWS as (i) having the best GUI to benefiting user interaction, (ii) the easiest platform to use, and (iii) the preferred cloud platform for creating VMs. Once confirmed by independent replications, our results suggest that AWS outperforms Azure regarding QiU. Therefore, cloud beginners might have a better cloud experience starting off their cloud projects by using AWS rather than Azure. In addition, our results may help to explain the AWS's cloud leadership.

CCS CONCEPTS

• **Computer systems organization** → **Cloud computing**; • **General and reference** → *Experimentation*; • **Social and professional topics** → *Quality assurance*;

KEYWORDS

Cloud platforms, Quality in Use, Experimentation

*Corresponding author.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
CASCON'18, October 2018, Markham, Ontario, Canada
© 2018 Copyright held by the owner/author(s).

ACM Reference Format:
Gabriel Costa Silva, Reginaldo Ré, and Marco Aurélio Graciotto Silva. 2018. Evaluating Efficiency, Effectiveness and Satisfaction of AWS and Azure from the Perspective of Cloud Beginners. In *Proceedings of 28th Annual International Conference on Computer Science and Software Engineering (CASCON'18).* ACM, New York, NY, USA, 12 pages.

1 INTRODUCTION

Making choices is central in software development. Choosing a subset of agile practices, programming languages, and development tools are only a few examples. Whereas the development context might drive several choices, other choices might require thorough analysis. For instance, it is not straightforward for cloud beginners to choose a cloud platform to deploy their projects on.

In this study, a cloud beginner includes those who have not yet deployed cloud applications into production, but are planning to, and those who are new to cloud computing and still lacks experience. Cloud beginners are critical to the cloud market because they currently correspond nearly to one-third of cloud users [44].

As a beginner, one does not have sufficient experience to make rigorous assessment on which cloud platform to use. In this case, one alternative is to go for the leader in the market. However, a thin line may separate first places in the cloud market [19]. Other alternative is to choose the cloud platform that offers the largest service set. But, this also offers little clue as even different cloud platforms provide similar services [36]. Finally, since the cloud market is cost-driven [13], one could choose a cloud platform based on the lowest price. Nevertheless, this would be tricky as prices slightly differ among cloud platforms[1].

The alternative we investigate in this paper (i.e., our research question) is whether cloud platforms significantly differ regarding their quality in use (QiU). The ISO/IEC 25010[2] defines QiU as *"the degree to which a product or system can be used by specific users to meet their needs to achieve specific goals with effectiveness, efficiency, freedom from risk and satisfaction in specific contexts of use."* Identifying differences regarding QiU could drive cloud beginners to the cloud platform that offers the best cloud experience.

Unlike usability that is a *quality attribute* in the product quality model, QiU is a *quality model* that consists of five attributes. Both the product quality model and the QiU model are defined by the ISO/IEC 25010. As its predecessor, the ISO/IEC 9126, this standard is intended to ensure the quality of software-intensive products [2].

[1] https://aws.amazon.com/ec2/pricing/, https://azure.microsoft.com/en-us/pricing/
[2] https://www.iso.org/obp/ui/#iso:std:iso-iec:25010:ed-1:v1:en

QiU is key for successful software adoption. For instance, an industrial survey shows the ease of use as the second most important criterion when evaluating a development technology, and the same survey shows effectiveness as the top driver for technology adoption [34]. Furthermore, another survey shows that efficiency is deemed as an essential goal for delivering software systems [40].

Different from several studies that analyse external quality attributes of cloud platforms [32], this study uses concepts and techniques from software quality, empirical research and cloud computing to analysing two public cloud platforms. Our goal is evaluating their QiU from the point of view of cloud beginners. This evaluation takes into consideration the context of undergraduate students creating virtual machines (VMs) on Amazon Web Services (AWS) and Microsoft Azure (Azure) platforms.

Our results show that participants where more efficient performing tasks on AWS than on Azure ($p = 0.000012$, $V = 617$, $\gamma = 0.68$, power = 0.96). We observe that these results are consistent across trials and participants, apart from a few exceptions. Likewise, most participants declared to be more satisfied with AWS than with Azure regarding the interaction with graphical user interface (GUI), ease of use and preference for virtual machine creation.

Like efficiency and satisfaction, participants also achieved greater effectiveness on AWS than on Azure. However, we found several issues that require further studies on effectiveness. For instance, we did not identify a statistically significant relationship between platform and task effectiveness ($p = 0.1396491$), and we observed that effectiveness of trials and participants varied with the platform. These results are detailed in Section 4.

As we discuss in Section 5, our conclusion that *"cloud platforms may significantly differ regarding a subset of QiU attributes, such as efficiency"* should be considered as preliminary. We explain that our findings are indicative and we do not claim for comprehensiveness or generalisation. As any other, our study dealt with several threats to validity that are explained in Section 6.

To achieve sound results, we thoroughly planned and conducted our experiments following rigorous guidelines, as we detail in Section 3. Section 2 overviews concepts of software quality and cloud computing that we used to conduct this study. Finally, Section 7 concludes the paper and summarises an agenda for future work.

2 BACKGROUND

In this section, we explain the importance of using a quality model to define quality attributes. Next, we overview models for product quality and for quality in use that are defined by the ISO/IEC 25010 standard - the most used way to define quality in software systems. Then, we discuss the common use of experimentation to evaluate quality of software systems.

We then move to the topic of cloud computing to present the entities that form the cloud computing model. Understanding this model is key to analyse the design and the results of our study.

2.1 Software Quality

According to Sommerville [52], software quality management is concerned with *"ensuring that the required level of quality is achieved in a software product." Quality* is often represented by a set of quality attributes. Quality attributes (QAs) are external characteristics of a software system, or are results of the interactions between users and systems [25]. Performance, usability and security are common examples of QAs [3].

Quality Models (QMs) are used to (i) define, (ii) assess and (iii) predict QAs [54]. This section does not delve into concepts of predictive QM as they are out of the scope of this paper. Recently replaced by the ISO/IEC 25010[3], the ISO/IEC 9126[4] is the most used definitional QM [39]. Adopting a definitional QM is essential when working with QAs since QAs are often subjective [23] and different stakeholders may have different views for the same QA [3].

The ISO/IEC 25010 describes QMs for software product quality and software quality in use (QiU). Whereas software product quality refers to external QAs, such as a performance, usability or security, QiU refers to results of interactions between users and software systems in order to achieve specific goals [5]. The QiU model defines five QAs: (i) effectiveness, (ii) efficiency, (iii) satisfaction, (iv) freedom from risk, and (v) context coverage. These QAs may be further decomposed into subcharacteristics. For instance, *context coverage* may be decomposed into context completeness and flexibility.

An assessment model can be used to evaluate QAs. Evaluating QAs is critical to ensure that the application achieves the desired quality level [9]. Assessment models extend quality definition models (e.g., ISO 25010) by proposing metrics that evaluate QAs [54]. However, a common approach to evaluate QAs is not by using quality assessment models, but by testing causal or correlational relationships [12] (i) between QAs and software characteristics, such as software design properties [45] and development paradigm [33], or (ii) between the QA and its related process, such as a task in the maintenance process (e.g., testing, fixing bugs) [22].

However, evaluating QAs is difficult because they depend on multiple environmental factors [16], which substantially increases the set of variables to be analysed. As an example, software performance cannot be measured without first quantifying the processor and memory attributes of the computer on which the experiments are carried out. One strategy to isolate variables and measure the causal relationship between the variable of interest and the QA is by using experimentation [12].

Experimentation has enabled the study of QAs such as maintainability and changeability, improving the understanding of the software characteristic under study [6, 25], and allowing its measurement and prediction [27, 43]. Testing causal relationships by experimentation is extensively used in Section 4 to investigate whether cloud platforms significantly differ regarding their QiU. The next section clarifies key concepts of cloud computing that are target of our experiments in Section 4.

2.2 Cloud Computing

The National Institute of Standards and Technology (NIST) defines cloud computing as *"a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction"* [35].

[3] http://iso25000.com/index.php/en/iso-25000-standards/iso-25010
[4] http://www.iso.org/iso/catalogue_detail.htm?csnumber=22749
[5] https://www.iso.org/obp/ui/#iso:std:iso-iec:25010:ed-1:v1:en

In Silva et al. [49], we present the main entities that form the cloud computing model. Figure 1 illustrates these entities by using an UML class diagram. A cloud *Platform* provides *Service*, such as computing and storage. A cloud *Platform* has a name, such as Amazon Web Services. A *Service* might be managed through multiple *Management Interfaces*, which are provided by cloud *Platform*. A *Management Interface* has a type, such as graphical user interface (GUI), and properties, such as authentication attributes.

The *Platform* is responsible for the *Cloud User* management. A *Cloud User* is identified by its name, and can have several keys to access its *Resources*. A *Service* is identified by its name, such as EC2, or Cloud Servers. A *Service* might be supported by other service. For example, Amazon EBS and S3 supports Amazon EC2, providing persistent storage. Each *Service* might operate in a different *Region*.

A *Region* represents a wide geographical location, such as Europe or Asia. In addition to its name, a *Region* might have a particular code assigned by the cloud *Platform*. This code is represented in this conceptual model by the *url* attribute. A *Region* might have several *Datacentres*, which are identified by their names and urls.

A *Service* has *ServiceOperations*. A *ServiceOperation* represents those management operations that a *Cloud User* can perform on its *Resource* through a *Management Interface*, such as sending a message to a queue in a message queue service. A *ServiceOperation* is identified by its name, such as runInstance or listImages, for example. Some operations rely on input parameters, such as the name of image, and region. These operations have a single return value, which might be a list of results, for example.

A *Service* exists to provide *Resources*, such as VMs and storage containers. Each *Platform* might define different properties and states for *Resources* provided by its *Service*. However, two properties are present in most of *Resources*: id and name. Once the *Resource* has been created, it is made accessible by one or more *Endpoints*. The *Resource* is available in one of the *Region* supported by the *Service*. One *Resource* might support another. This is the case of storage containers, which are used to support a set of files (*Resource*).

Some *Resources* rely on *OperationalResource*. An *OperationalResource* represents internal resources provided by the *Service*, such as the hardware of a VM, or the engine of a database service. In addition to the *Cloud User* credentials, some services provide further *Security Mechanism*, such as firewall and permissions.

A *Security Mechanism* consists of a set of *Security Entry*. For example, a file in a storage service might have different permissions (*SecurityEntry*). Each *SecurityEntry* is assigned to the *Resource* it protects. More sophisticated mechanisms require further elements, represented in the meta-model by *Security Rule*. This is the case of security groups, provided by Amazon EC2.

3 STUDY PLAN & EXECUTION

Whereas previous section presents key concepts to understand our study, this section details how we synthesised those concepts to undertake our study. This study consists of three experiments aiming to analyse two cloud platforms for evaluating their quality in use (QiU) from the point of view of cloud beginners, in the context of undergraduate students creating virtual machines (VMs) on Amazon Web Services (AWS) and Microsoft Azure (Azure) platforms.

As explained in Section 2.1, experimentation is a common strategy to identify factors that impact a particular QA [1, 6, 11, 12, 25]. Therefore, this is the strategy used in this study to identify whether cloud platforms significantly differ regarding their QiU.

Although there are several guidelines for conducting and reporting experiments in software engineering [26, 29, 30, 50], we used the framework proposed by Wohlin et al. [55] as it enables rigorous experiment preparation, execution, analysis and reporting.

In addition, this study is mainly based on the literature of software maintainability [7, 22, 33, 41] to make many design decisions. The rationale for this is that this study was performed in the context of software quality. Literature reviews on software quality show that software maintainability is one of the most empirically studied quality attribute of software systems [25, 39]. Therefore, we assume these studies as appropriate references for carrying out our study.

Next sections correspond to elements of the experimentation framework we used, detailing the protocol for the experiment and its execution. Terminology associated with the experimentation framework is highlighted when it first appears in the main text.

3.1 Task

Only one *task* was performed in this study. The task consists of manually creating a VM by using the Graphical User Interface (GUI) provided by cloud platforms. The complexity of our experimental task may vary with the configuration required by the VM.

For instance, accepting the default configuration suggested by the cloud platform is as easy as just clicking on "next" buttons. However, cloud applications might require more elaborated configurations, such as a specific operating system version or a powerful hardware type. In order to measure whether the QiU varies with the configuration, we performed 7 *trials* as detailed in Table 1.

Cloud services may offer significantly different features [36], which directly impact on the their configuration complexity. Therefore, to defining the configuration set for our trials, we based on our previous work on abstracting differences among cloud services [48]. Operating system, security group, region and hardware type are common features across different cloud computing services.

We did not use any Windows operating system because of (i) our limited financial resources impeded using high cost configurations, such as Windows; and (ii) boot times for Windows VMs are often longer than for Linux VMs, which would exceed the allocated time for our experiments. Possible threats to validity due to our configuration choices are discussed in Section 6.

3.2 Selection of Variables

Independent variable. Cloud platform is the *independent variable* in all three experiments we carried out. However, defining our *treatments* is not straightforward as there is a plethora of cloud platforms in the cloud market.

We chose Amazon Web Services (AWS) and Microsoft Azure (Azure) as our treatments because of their leadership in the cloud market [44]. We also preferred only two treatments to keep both the *experimental design & analysis* simple [55]. The independent variable is measured in a *categorical* scale, whereby each cloud platform represents a category.

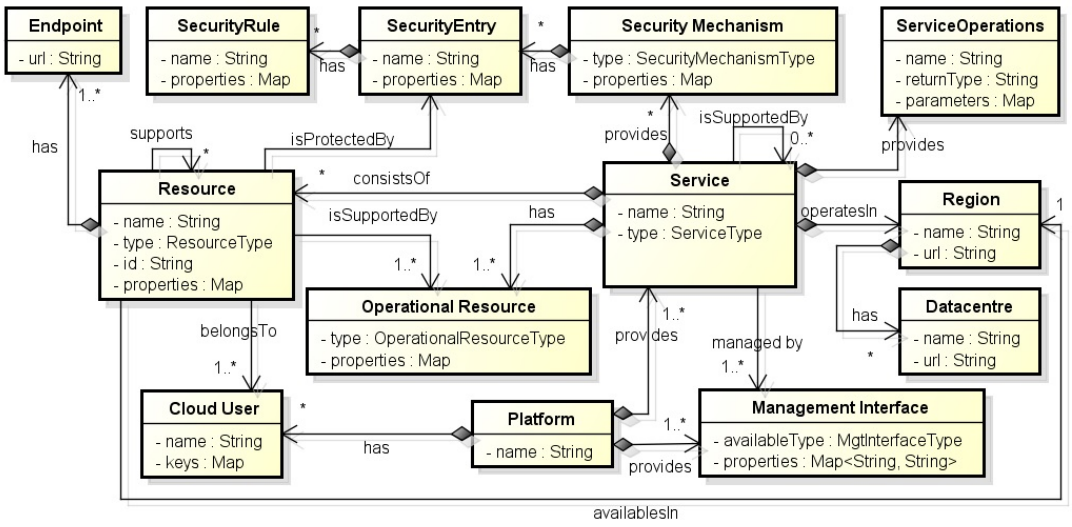


Figure 1: Entities that form the cloud computing model, as presented in Silva et al. [49].

Table 1: We performed seven *trials* to evaluate whether the QiU varies with the configuration.

Trial #	VM Requirements				
	OS - version	Security Group	Region	Hardware	Bootstrap script
01	Ubuntu - 16.x	defaultSG	Default	Cheapest	None
02	Ubuntu - 16.x	defaultSG	Default	2 <vCPUs <4, 2GB <RAM <8GB	None
03	Ubuntu - 16.x	defaultSG	Alternative	Cheapest	None
04	Ubuntu - 16.x	defaultSG	Default	Cheapest	Install & start tomcat7
05	Ubuntu - 16.x	New: allow TCP on port 8080	Defaul	Cheapest	None
06	SUSE - any version	defaultSG	Default	Cheapest	None
07	Ubuntu - 14.x	New: allow TCP on port 8181	Alternative	2 <vCPUs <4, 2GB <RAM <8GB	Install & start tomcat7

However, AWS and Azure are huge platforms and therefore, evaluating the entire platforms with a single study is an unrealistic goal. As usual in software quality studies [25], we chose a single cloud service provided by both platforms to act as a proxy for evaluating the entire platform. We chose the computing service because of (i) their popularity [19] and (ii) their similar configuration set across platforms (see Section 3.3). Note that this choice limits our study, which we discuss in more detail in Section 6.

Dependent variable. Efficiency, effectiveness and satisfaction are the *dependent variables* in our study. Each dependent variable was used for a single experiment. We preferred this strategy as adding several dependent variables to a single experiment would increase both the *experimental design & analysis* complexity [55].

We chose efficiency, effectiveness and satisfaction because they are part of the QiU definition (Section 2.1). Note that we limited our definition of QiU to these three quality attributes, excluding from our study the concepts of "freedom from risk", and setting the "context coverage" to a single known context.

Although "freedom from risk" and "context coverage" are also part of the QiU model, like efficiency, effectiveness and satisfaction, they require different *experimental tasks & design* to be evaluated. Moreover, they are out of our research interests.

We measure efficiency as the time taken, in seconds, to perform the experimental task. Participants were responsible for taking note of begin and end times by using an online spreadsheet. The begin time is the moment when the participant starts the new VM configuration whereas the end time is the moment immediately after the participant finishes requesting the new VM. Therefore, we neither take into account the VM boot time nor the time required to sign in to the platform. Efficiency is measured on a *ratio* scale, whereby values vary from 0 to the total of seconds taken.

Effectiveness is measured as whether the task was successfully performed. In our experiment, "successfully performed" means that

participants did not face problems instantiating VMs due to incorrect configuration or procedure. Participants were responsible for taking note whether VMs instantiated properly by using the same online spreadsheet used to control efficiency. This required participants to wait VM instantiation in order to check for effectiveness. The experimenter randomly checked for inconsistencies between data collected by participants and log data collected by cloud platforms, but no inconsistencies were found. Efficiency is measured on a *categorical* scale, whereby valid values are "yes" or "no" meaning the success on performing the task.

Unlike efficiency and effectiveness, satisfaction was not inferred from results of performing a task, but from results of a questionnaire applied after the experiment. To measure satisfaction, we asked participants three questions. The first question asks which cloud platform's graphical user interface (GUI) best benefits user interaction. The second question asks which is the easiest platform to perform the experimental task. The third question asks which cloud platform the participant would prefer. For the three questions, satisfaction is measured on a *categorical* scale, whereby valid values are "AWS" or "Azure" meaning the cloud platform chosen.

Possible threats to validity due to our choices on dependent variable measures are discussed in Section 6.

3.3 Instrumentation

AWS EC2 and Azure Virtual Machines are the two main *instruments* used in this study. As explained in Section 3.2, they were used as proxies to evaluate cloud platforms. AWS EC2 and Azure Virtual Machines are computing services intended to managing VMs in the cloud [36]. Whereas for AWS EC2 we used an academic account to have access to the service, for Azure Virtual Machines we used a commercial account in the free period. Both accounts allowed full access to all computing service features.

Usually, cloud platforms provide more than one interface for accessing their services [49], such as GUI, Application Programming Interfaces (APIs), and command line tools. In this study, we chose the GUI as the *instrument* for creating VMs because most cloud users use it to manage and automate workloads in cloud environments [19]. Section 6 discusses threats to validity due to this choice.

Data collection form and two questionnaires were also used as *instruments* in this study. These instruments are available online[6] to encourage future replications of this study.

3.4 Participant Selection

A group of eleven third-year undergraduate students in software engineering took part in this study. They were enrolled at the Universidade Tecnológica Federal do Paraná, in Dois Vizinhos, Brazil. The researcher was a lecturer at this undergraduate course, which prompted the choice. The student selection was driven by *convenience*, according to the student interest in participating.

Because this study aims to investigate cloud beginners, limited cloud knowledge was a mandatory criterion to selecting students. Students had the freedom to choose whether or not to participate, without any risk of penalty to their undergraduate course. However, no participant gave up (*drop-out = 0*). Each participant received a bonus grade for their participation. Possible threats to validity due to our choices are discussed in Section 6.

[6]https://github.com/EmpiricalStudies/qiu-cloud-platforms-2018

3.5 Study Design & Hypothesis Definition

Since an experiment measures a single cause-effect relationship [55], we carried out three experiments – one for each dependent variable. Each experiment consists of one *factor*, and two *treatments*, as presented in Section 3.2.

Two hypotheses were defined for each experiment. The *null hypothesis* states that the summary statistic of a dependent variable is the same regardless of the cloud platform used whereas the *alternative hypothesis* states the opposite.

The summary statistic is computed by taking each *data point* in a *dataset*. As datasets consist of several data points, the summary statistic is calculated by using median for efficiency, correctness rate for effectiveness, and response rate for satisfaction. As we explain in Section 3.6, the median was preferred instead of the mean because it is more robust.

Due to the limited number of participants, we decided for a *within-subject* design. In practice, it means that all eleven participants worked on both *control* and *experimental* groups. In addition, we randomly sorted trials and platforms before assigning them to each participant in order to increase *experimental validity*.

3.6 Data Analysis

We adopted only *non-parametric* statistics and tests in this study, as summarised in Table 2. The rationale is that (i) *non-parametric* statistics are less sensitive to *outliers* and (ii) *non-parametric* tests make no assumption about the data distribution [5, 28]. Andrews et al. [4] underpinned our statistical test selection. Foundation for statistical tests we used can be found in [21, 55]. Charts are used to support data visualisation.

The *Type I error*, also known as *significance level* (α), is the probability of falsely rejecting the null hypothesis [55]. In our experiments, the α was set up to traditional 0.05 as we had no reason to increase or decrease this value. This α level implies on a *confidence level* of 0.95 when accepting the null hypothesis.

We also calculated the *effect size* (γ). The γ is important because it measures the magnitude of the difference between experimental treatments [38]. In this study, the Vargha & Delaney A measure [53] is used to measure the γ for efficiency whereas odds ratio [21] was used for effectiveness.

The *A*-statistic, produced by the A measure, is a value between 0.5 (no effect) and 1 (large effect). A value less than 0.5 should be subtracted from 1 to obtain the effect size. For instance, if the *A*-statistic is 0.35, then the effect size is 0.65 – which is considered medium, according to [53].

The statistic produced by odds ratio is a ratio value meaning the odds of a treatment outperforms the other a number of times [10]. For example, an odds ratio of 3.35 means that the odds of treatment X outperforms treatment Y is 3.35 times higher. As Breaugh [10] explains, a value of 1.00 means no relationship between treatments. Furthermore, odds ratio is symmetric around 1.00. Therefore, the strength of association increases as the odds ratio depart from 1.00.

Table 2: Summary of statistics and statistical tests adopted in this study.

Statistic Type	Statistic/Test	Definition/Config.	Purpose
Descriptive	Median and quartiles	25%, 50%, 75%, 100%	To identify the middle value and variance.
	Range	Maximum - Minimum	To quantify the data dispersion.
	Response rate	0% - 100%	To quantify the share of a variable in the total.
Inferential	A-test	A-statistic	To identify the magnitude of the difference between treatments for a ratio-measured dependent variable.
	Wilcoxon signed-rank	Two-tailed	To test the statistical significance of differences between two medians.
	Fisher's exact test	2x2 contingency table	To identify whether there is a relationship between two categorical variables in a small sample.
	Odds ratio	2x2 contingency table	To identify the magnitude of the difference between treatments for a categorical-measured dependent variable.

The R statistical software[7] was used for performing data analysis. The package *effsize*[8] was used to calculate the effect size.

3.7 Experiment Execution

A pilot experiment was performed to check the task complexity, training and experimental materials, and time required for performing the task. The pilot experiment was undertaken by a group of 6 undergraduate students that did not take part in the official experiment. The result of this pilot prompted some changes, leading to the current protocol detailed in this paper. In addition, it provided preliminary guesses on the results. The experiment was undertaken in four steps: (i) participant selection; (ii) training; (iii) experimentation; and (iv) feedback.

(1) *Participant selection.* The process for participant selection was previously described in Section 3.4. After selection, participants received an *introduction script*. This script provided an overview for the experiment, highlighting its purpose, the task that participants should perform, and benefits of taking part in the experiment. Those who accepted taking part in the training session were asked to fill out the *participant characterisation form*. This form collected information regarding the participants' background, such as professional experience. Questions on this form were based on guidelines proposed in [20];
(2) *Training.* As participants had little or no experience on cloud computing (because they are cloud beginners), a training session was necessary to present basic concepts related to cloud and cloud platforms. Moreover, it prepared participants to perform the task evaluated in this study. The total training time was of eight hours;
(3) *Experimentation.* Just before starting the experiment, participants were asked to sign in the *participant consent form*, consenting in taking part in the experiment. Next, each participant received a unique number that was used to identify them throughout the experiment. The experiment consisted in performing the experimental task (Section 3.1) for the two cloud platforms (Section 3.3) while controlling the time spent for the task. We created individual accounts for each participant to enable them to freely interact with the cloud service without interference from other participants. For example, to start a task a participant did not need to wait till another participant finishes their task. Each individual account was linked to the main account created by the researcher. We limited the permissions of individual accounts to the computing service, as this service was the focus of our study. During the experiment execution, participants were free to get in and out of the room, e.g. go to the toilet, as long as this time off was not counted as part of the task performed in the experiment. The experimenter observed the experiment execution without interfering in tasks performed by participants. The experiment was performed in two hours;
(4) *Feedback.* When the experiment had finished, participants were asked to fill out a *feedback form*, assessing their experience in this experiment. Participants received participation certificates a few weeks after submitting the *feedback form*.

[7] http://www.r-project.org
[8] http://cran.r-project.org/web/packages/effsize/index.html

4 RESULTS

This section presents the results of applying the protocol presented in Section 3 whereas Section 5 examines the impact of these results on hypothesis testing and research questions, and discuss implications of our findings.

4.1 Data Reduction

The original dataset consisted of 154 data points. However, participant #11 did not inform the effort for task #3 on Azure, and participant #10 did not inform the correctness of task #4 on AWS. Therefore, we removed these two data points from our analysis, which resulted in 152 data points – 76 for each platform.

4.2 Demographics

Our participants were 19 (18.2%), 20 (27.3%), 21 (9.0%), 22 (27.3%) and 23 (18.2%) years old. Whereas 81.8% of our participants identified themselves as males, 18.2% preferred not to answer.

When asked to classify their current knowledge on cloud computing from 0 (no knowledge at all) to 5 (expertise), most participants (54.5%) scored their knowledge as 1, four participants (36.6%) scored their knowledge as 2, and only one (9%) scored their knowledge as

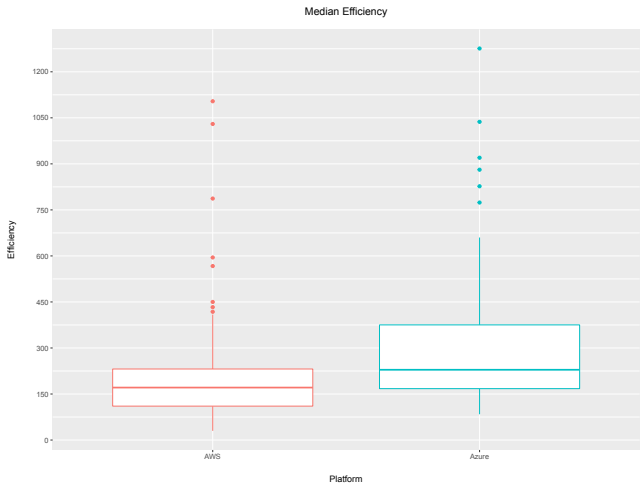


Figure 2: Efficiency of participants on performing the task on cloud platforms. Measured in seconds.

3. Regarding their experience creating VMs for their personal use or work, three participants (27.2%) had experience on both AWS and Azure whereas five participants (45.4%) had no experience at all. Whereas two participants (18.2%) had experience on Azure only, one participant (9%) had experience on AWS only.

4.3 Efficiency Analysis

Participants performing tasks on AWS ($\tilde{x}$ = 171.0 secs) were 58 secs (25.3%) more efficient than on Azure ($\tilde{x}$ = 229.0 secs) on average (Figure 2). The Wilcoxon test reveals that this difference is statistically significant at α = 0.05 (p = 0.000012, V = 617).

In practice, it means that the probability that the efficiency difference between platforms was achieved only by chance is discarded for this experiment. In addition, the medium effect size (γ = 0.68) and high statistical power (power = 0.96) suggest that the difference between platforms is significant.

Table 3 shows that AWS outperformed Azure in efficiency for the seven trials performed, with differences varying from -40 secs for trial #4, to -191 secs for trial #7. Apart from trial #2 (γ = 0.62) and trial #7 (γ = 0.60), all trials achieved a medium (0.64 $< \gamma \leq$ 0.71) or large ($\gamma >$ 0.71) effect size. Similarly, individual results of participants also shown that AWS was more efficient than Azure, apart from results of participants #1 and #4 (Table 4).

4.4 Effectiveness Analysis

The table in Figure 3 shows that tasks performed on AWS (correctness = 92.1%) were more effective than on Azure (correctness = 82.8%). Although the Fisher's exact tests did not identify a statistical significant relationship between platform and task effectiveness (p = 0.1396491), the effect size was relevant (γ = 0.41).

Table 5 shows that trials #4 and #7 presented the smallest effectiveness for AWS whereas trials #5 and #6 presented the smallest effectiveness for Azure. Although we can observe some differences in effectiveness between platforms for some trials, we could not identify any pattern that explains the differences.

```
##   Cell Contents
## |-------------------------|
## |                       N |
## |              Expected N |
## | Chi-square contribution |
## |           N / Row Total |
## |           N / Col Total |
## |         N / Table Total |
## |-------------------------|
##
## Total Observations in Table:  152
##
##              | correct
##     platform |        no |       yes | Row Total |
## -------------|-----------|-----------|-----------|
##          AWS |         6 |        70 |        76 |
##              |     9.500 |    66.500 |           |
##              |     1.289 |     0.184 |           |
##              |     0.079 |     0.921 |     0.500 |
##              |     0.316 |     0.526 |           |
##              |     0.039 |     0.461 |           |
## -------------|-----------|-----------|-----------|
##        Azure |        13 |        63 |        76 |
##              |     9.500 |    66.500 |           |
##              |     1.289 |     0.184 |           |
##              |     0.171 |     0.829 |     0.500 |
##              |     0.684 |     0.474 |           |
##              |     0.086 |     0.414 |           |
## -------------|-----------|-----------|-----------|
## Column Total |        19 |       133 |       152 |
##              |     0.125 |     0.875 |           |
## -------------|-----------|-----------|-----------|
##
## Fisher's Exact Test for Count Data
## ----------------------------------------------------------------
## Sample estimate odds ratio:  0.4177346
##
## Alternative hypothesis: true odds ratio is not equal to 1
## p =  0.1396491
## 95% confidence interval:  0.1225451 1.263983
```

Figure 3: Results of Fisher's exact test for effectiveness of tasks performed on cloud platforms.

Table 6 shows the effectiveness by participant. Participants #2, #5 and #6 had several issues when performing their tasks on both AWS and Azure. From results in Section 4.2, we note that participant #2 had no previous experience creating VMs on both platforms whereas participants #5 and #6 had previous experience on Azure.

4.5 Satisfaction Analysis

The AWS GUI best benefits user interaction for 63.7% of our participants whereas 36.3% of our participants claim the same for Azure. We checked whether satisfaction with GUI correlates with previous experience on the platform (Table 7), but we could not identify any significant correlation neither for AWS nor for Azure.

AWS is the easiest platform to perform the experimental task for 81.8% of our participants. One participant (9.09%) could not observe significant differences between platforms whereas another participant chose Azure as the easiest. As we did for satisfaction with GUI, we checked whether the choice of the easiest platform correlates with previous experience on the platform, but we could not identify any correlation as two-thirds of those who chose AWS as the easiest platform had no previous experience on AWS.

Finally, 81.8% of our participants prefer AWS rather than Azure for creating VMs. Again, we could not identify any correlation between preference and previous experience. For instance, 55.5% of those who prefer AWS for creating VMs had no previous experience

Table 3: Statistics of efficiency by trial.

Trial #	Platform	Min	1st Qu.	Median	3rd. Qu.	Max.	Δ Median	Δ %	Effect Size
1	AWS	30.0	87.0	138.0	199.5	567.0	-43	23.7	0.66
	Azure	84.0	151.5	181.0	212.0	288.0			
2	AWS	109.0	171.0	183.0	270.0	787.0	-61	25.0	0.62
	Azure	107.0	191.5	244.0	346.5	1276.0			
3	AWS	42.0	106.5	142.0	176.0	433.0	-55	27.9	0.70
	Azure	111.0	148.8	197.0	326.8	881.0			
4	AWS	93.0	129.5	180.0	219.0	290.0	-40	18.1	0.70
	Azure	150.0	185.5	220.0	265.0	605.0			
5	AWS	64.0	101.0	180.0	249.0	595.0	-96	34.7	0.76
	Azure	150.0	233.5	276.0	469.0	827.0			
6	AWS	60.0	77.5	120.0	169.5	400.0	-100	45.4	0.80
	Azure	127.0	160.5	220.0	339.5	534.0			
7	AWS	120.0	175.5	236.0	434.0	1104.0	-191	44.7	0.60
	Azure	140.0	220.5	427.0	716.0	1037.0			

Table 4: Statistics of efficiency by participant.

Participant #	Platform	Min	1st Qu.	Median	3rd. Qu.	Max.	Δ Median	Δ %	Effect Size
1	AWS	150.0	182.5	210.0	290.0	450.0	47	28.8	0.62
	Azure	152.0	160	163.0	260.5	357.0			
2	AWS	120.0	133.0	195.0	306.0	400.0	-81	29.3	0.75
	Azure	140.0	223.0	276.0	529.5	881.0			
3	AWS	85.0	171.0	216.0	524.5	1030.0	-84	28.0	0.67
	Azure	200.0	221.0	300.0	496.5	1276.0			
4	AWS	108.0	130.0	220.0	230.0	400.0	10	4.7	0.54
	Azure	145.0	170.0	210.0	223.5	660.0			
5	AWS	70.0	103.0	109.0	127.5	150.0	-122	52.8	1
	Azure	169.0	197.5	231.0	242.5	462.0			
6	AWS	175.0	189.0	290.0	514.0	1104.0	-36	11.0	0.56
	Azure	219.0	235.5	326.0	415.5	827.0			
7	AWS	93.0	118.0	144.0	189.5	222.0	-59	29.0	0.69
	Azure	106.0	130.5	203.0	377.0	774.0			
8	AWS	30.0	53.0	69.0	230.0	418.0	-196	73.9	0.77
	Azure	143.0	162.0	265.0	323.0	920.0			
9	AWS	149.0	168.5	207.0	314.5	567.0	-74	26.3	0.51
	Azure	113.0	180.0	281.0	333.5	564.0			
10	AWS	77.0	96.0	108.0	165.0	236.0	-126	53.8	0.85
	Azure	127.0	194.5	234.0	386.0	1037.0			
11	AWS	55.0	70.0	90.0	136.5	201.0	-52.5	36.8	0.73
	Azure	84.0	114.0	142.5	357.8	605.0			

on AWS. In addition, from the two participants who preferred Azure, one had previous experience on Azure whereas the other had not.

5 DISCUSSION

This section discusses the impact of findings in Section 4 on our hypothesis (Section 5.1) and research question (Section 5.2). Furthermore, we explain our perspective on the validity and comprehensiveness of our findings (Section 5.3).

5.1 Hypothesis Testing

This study investigates three *null hypotheses* (Section 3.5) stating that efficiency, effectiveness and satisfaction are the same regardless of the cloud platform used.

The experiment in Section 4.3 evaluated the efficiency by analysing the median time participants spent to carry out a task on AWS and Azure cloud platforms. As results show, participants took, on median, less time to perform tasks on AWS than on Azure. Although the range time took on each cloud platform overlaps broadly, the p-value is significant at $\alpha = 0.05$ and the effect size is considered

Table 5: Effectiveness by trial. Columns present the # of trials performed correctly or not on each platform, followed by the correctness rate.

Trial #	AWS			Azure		
	No	Yes	%	No	Yes	%
1	0	11	100%	1	10	90.9%
2	1	10	90.9%	1	10	90.9 %
3	0	11	100%	2	8	80%
4	2	8	80%	1	10	90.9%
5	1	10	90.9%	3	8	72.7%
6	0	11	100%	3	8	72.7%
7	2	9	81.8%	2	9	81.8%

Table 6: Effectiveness by participant. Columns present the # of tasks performed correctly or not on each platform, followed by the correctness rate.

Participant #	AWS			Azure		
	No	Yes	%	No	Yes	%
1	0	7	100%	0	7	100%
2	2	5	71.4%	6	1	14.2%
3	0	7	100%	2	5	71.4%
4	0	7	100%	0	7	100%
5	1	6	85.7%	2	5	71.4%
6	3	4	57.1%	1	6	85.7%
7	0	7	100%	0	7	100 %
8	0	7	100%	0	7	100%
9	0	7	100%	0	7	100%
10	0	6	100%	2	5	71.4%
11	0	7	100%	0	6	100%

Table 7: Satisfaction regarding GUI compared with participants' previous experience on the platform.

GUI benefits UI	Platform	Result	Previous Experience on the Platform	
			Yes	No
best	AWS	Yes	4	3
		No	0	4
	Azure	Yes	1	3
		No	4	3

medium. Therefore, we **reject** the *null hypothesis* for efficiency and **accept** the *alternative hypothesis* that states that the efficiency to perform a task varies with the cloud platform used.

The experiment in Section 4.4 evaluated the effectiveness by analysing the correctness with which participants performed a task on AWS and Azure cloud platforms. Like the experiment on efficiency, participants had a higher correctness rate on AWS than on Azure. However, unlike the experiment on efficiency, for this experiment the difference between platforms was not found significant at $\alpha = 0.05$ although the effect size might be considered relevant according to the literature [10]. A possible cause for this result is the small sample, which we discuss in Section 6. In addition, we found several issues that need further study, such as differences in trials and participants. Therefore, we **failed to reject** the *null hypothesis* for effectiveness that states that the correctness is the same regardless of the cloud platform used.

The experiment in Section 4.5 evaluated the satisfaction by asking participants their perceptions on the cloud platform (i) that offers the best GUI to benefit user interaction; (ii) easiest to use; and (iii) preferred by participants. As results show, most participants preferred AWS for all three questions. As we could not find any evidence that could bias this result towards previous experience of participants, we **reject** the *null hypothesis* for satisfaction and **accept** the *alternative hypothesis* that states that the satisfaction of a participant varies with the cloud platform used. However, unlike the two previous experiments, this one does not use any inferential statistics to measure the significance of results.

5.2 Research Question

Our research question asks whether cloud platforms significantly differ regarding their quality in use (QiU). This investigation is important since identifying QiU differences could drive cloud beginners to the cloud platform that offers the best cloud experience. Therefore, it was important to recruit participants with little knowledge and experience on cloud platforms. As Section 4.2 shows, our participants fill this requirement although future replications of this study should consider older participants and greater sample.

Section 5.1 shows that two out of three null hypotheses tested where rejected. However, we noted that no statistical analysis was used to evaluate the null hypothesis on satisfaction. Therefore, taking only the results and the context of this present study into consideration, the answer for our research question is that *cloud platforms may significantly differ regarding a subset of QiU attributes, such as efficiency.*

5.3 Should a Cloud Beginner Avoid Using Azure?

Our results *do not* mean that cloud beginners should avoid a particular cloud platform, but they highlight the importance of evaluating attributes other than popularity, range of services available and price. In particular, one should evaluate how our findings may impact them. For example, Section 4.3 shows that the two cloud platforms differ on about 1 minute when regarding efficiency. Whereas this difference may suggest that one platform is more appropriate for cloud beginners than the other platform, it is up to the regular cloud user to evaluate how critical this difference is to them.

Finally, we should highlight that as long as QiU differences are confirmed by independent replications, a major implication of our findings is that they might help to explain AWS leadership.

5.4 Analysis and Future Work

Two aspects should be considered when analysing our findings. First, as any other [47], experiments reported in this paper constitute a single piece of evidence rather than a definitive proof. It would be unrealistic to expect that a single experiment would enable

wide generalisations in different contexts [8]. Therefore, independent replications are necessary to increase the confidence in these results and possibly enable their generalization [37]. Meanwhile, our findings should be considered as preliminary.

Second, as we explain in Sections 1, 2.1 and 3, this study lies in the context of software quality, focusing on quality in use (QiU) rather than usability or human-computer interaction (HCI). Although QiU, usability and HCI may share some characteristics, they present different perspectives for the same phenomenon. In our view, it would be similar to explaining gravity by using general relativity and by using quantum mechanics – both understand gravity in different ways [15]. Therefore, unless proven the contrary, our preliminary findings are valid only within the QiU perspective.

Finally, we envisage three important tasks as part of future work. First, one could investigate whether QiU differences hold over different cloud services and management interfaces. This would enable the generalisation of our results to the entire cloud platform rather than limit them to the current design. Second, one could extend this study to other relevant cloud platforms, such as OpenStack[9]. This would enable understanding whether QiU is as heterogeneous as internal implementations of cloud platforms are. Finally, once our results are confirmed by independent replications, it is essential to investigate the reasons that led to QiU differences.

6 THREATS TO VALIDITY

We took a set of measures to mitigate threats to validity although it was not always possible to achieve this goal. The measures we took include following common strategies used by other researchers, applying guidelines from the literature, and making available online all material used throughout this study[10]. This section discusses the main threats to validity for this study highlighting aspects we believe should be considered in future replications of this study.

6.1 Internal Validity

Threats to internal validity help to explain why cause-effect relationships may be incorrect [55]. In this section, we discuss three major internal threats we could identify in our study.

Section 3.2 explains that we used the computing service as a proxy for evaluating the QiU of cloud platforms. Although using a proxy is the common practice in software quality literature [25], results in Section 4 can change with a different proxy. As there are several cloud services that could be used as proxies (see Section 2.2), we used a popular cloud service to mitigate this threat.

As Section 2.2 shows, cloud platforms provide several interfaces for service management. Section 3.3 clarifies that we used only one interface in our study. The GUI was the most convenient interface since most cloud users use it to manage and automate workloads in cloud environments [19]. However, it is important to take into account that we are dealing only with one single aspect of the cloud platform (i.e., the interface) and our results refer to this single aspect, not to the entire platform.

Finally, the measures we used to represent our dependent variables (Section 3.2) may represent another threat to validity. Efficiency, effectiveness and satisfaction can be measured in several ways. The ISO/IEC 25010 provides guidance on how to measure these attributes. As we did not have access to the full document of ISO/IEC 25010, we followed common practices in the literature for defining measures for efficiency and effectiveness. For instance, time is a common measure for efficiency in cloud computing studies [31, 32, 42]. We used an assessment mechanism for checking effectiveness of tasks similar to those used by Arisholm and Sjøberg [7] and Freeman and Schach [22] in their experiments.

On the other hand, when measuring satisfaction, we adopted a different strategy from the literature. Satisfaction is often measured by using Likert scale questions in studies regarding human behaviour in software engineering [17]. However, as we were dealing with beginners, we analysed that they would not be able to provide a confident evaluation in a scale given their little experience. Therefore, we preferred for a single dichotomous response based on the cloud platforms evaluated.

6.2 External Validity

Threats to external validity help to explain why experimental outcomes may not hold over different participants, environments, and treatments [55]. In this section, we discuss two major external threats we could identify in our study.

We used students as a convenience sample to selecting our participants (Section 3.4). We highlight that this is a common practice for experiments in software engineering [14, 51], and that our participants fill the criteria of cloud beginners (see Section 4.2). As there is a long debate in the literature about the use of students in experiments [14, 24, 30, 51], we encourage that future replications of this study take professionals as participants to evaluate the extent to which these two types of participants differ in this matter.

Finally, eleven participants took part in this study. Compared with the subset of software engineering experiments that accounts only for undergraduate students, this is less than the average ($\tilde{x}$ = 43), but greater than the minimum (n = 10) [51]. The small sample may reduce the statistical power of inferential tests, and limit their generalisation. Although the statistical power in our experiment for efficiency (power = 0.96) was far greater than most experiments in software engineering (power = 0.36 when considered all type of tests, and power = 0.46 when considered only Wilcoxon) [18], our experiment for effectiveness failed to reject the null hypothesis. A possible reason for this failure is that the sample was not sufficiently large [46]. Therefore, increasing the sample is an aspect that should be considered in future replications of this study.

7 CONCLUSION

Our findings show that participants where more efficient performing tasks on AWS than on Azure (p = 0.000012, V = 617, γ = 0.68, power = 0.96). We observe that these results are consistent across trials and participants, apart from a few exceptions. Likewise, most participants declared to be more satisfied with AWS than with Azure regarding the interaction with GUI, ease of use and preference for VM creation although this finding is not backed by statistical test.

Like efficiency and satisfaction, participants also achieved greater effectiveness on AWS than on Azure. However, we found several issues that require further studies on effectiveness. For instance, we did not identify a statistically significant relationship between

[9] https://www.openstack.org
[10] https://github.com/EmpiricalStudies/qiu-cloud-platforms-2018

platform and task effectiveness (p = 0.1396491), and we observed that effectiveness of trials and participants varied with the platform.

Therefore, we concluded that *"cloud platforms may significantly differ regarding a subset of QiU attributes, such as efficiency."* As a result, AWS might be more appropriate for cloud beginners than Azure. However, we highlight that our findings are indicative and we do not claim for comprehensiveness or generalisation. As any other, experiments reported in this paper constitute a single piece of evidence rather than a definitive proof. Hence, independent replications are necessary to increase the confidence in these results and possibly enable their generalization.

Once our results are confirmed by independent replications, it is essential to investigate the reasons that led to these differences. Another important future work consists of extending this study to cover the CLI interface, and refine the experiment to thoroughly investigate effectiveness across different platforms.

ACKNOWLEDGMENTS

The authors are grateful to all students that took part in this study, and to Universidade Tecnológica Federal do Paraná that partially funded this study. This study is result of our research project on Legacy Software Modernisation in Micro & Small Companies, registered under the ID 2299 by the UTFPR – CP.

REFERENCES

[1] Jehad Al Dallal. 2013. Object-oriented class maintainability prediction using internal quality attributes. *Information and Software Technology* 55, 11 (nov 2013), 2028–2048. https://doi.org/10.1016/j.infsof.2013.07.005

[2] H. Al-Kilidar, K. Cox, and Barbara A. Kitchenham. 2005. The use and usefulness of the ISO/IEC 9126 quality standard. In *2005 International Symposium on Empirical Software Engineering, 2005.* IEEE, 122–128. https://doi.org/10.1109/ISESE.2005.1541821

[3] David Ameller, Claudia Ayala, Jordi Cabot, and Xavier Franch. 2012. How do software architects consider non-functional requirements: An exploratory study. In *2012 20th IEEE International Requirements Engineering Conference (RE).* IEEE, Chicago, IL, 41–50. https://doi.org/10.1109/RE.2012.6345838

[4] Frank M. Andrews, Laura Klem, Terrence N. Davidson, Patrick M. O'Malley, and Willard L. Rodgers. 1981. *A Guide for selecting statistical techniques for analyzing social science data* (2nd ed.). Survey Research Center, Institute for Social Research, University of Michigan, Michigan. 71 pages.

[5] Andrea Arcuri and Lionel C. Briand. 2011. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceeding of the 33rd international conference on Software engineering - ICSE '11.* ACM Press, New York, New York, USA, 1. https://doi.org/10.1145/1985793.1985795

[6] Erik Arisholm. [n. d.]. *Information and Software Technology* ([n. d.]). https://doi.org/10.1016/j.infsof.2006.01.002

[7] Erik Arisholm and Dag I.K. Sjøberg. 2004. Evaluating the effect of a delegated versus centralized control style on the maintainability of object-oriented software. *IEEE Transactions on Software Engineering* 30, 8 (aug 2004), 521–534. https://doi.org/10.1109/TSE.2004.43

[8] V.R. Basili, F. Shull, and F. Lanubile. 1999. Building knowledge through families of experiments. *IEEE Transactions on Software Engineering* 25, 4 (1999), 456–473. https://doi.org/10.1109/32.799939

[9] Len Bass, Paul Clements, and Rick Kazman. 2013. *Software Architecture in Practice* (3 ed.). Addison-Wesley, Boston, Massachusetts. 640 pages.

[10] J Breaugh. 2003. Effect Size Estimation: Factors to Consider and Mistakes to Avoid. *Journal of Management* 29, 1 (jan 2003), 79–97. https://doi.org/10.1016/S0149-2063(02)00221-0

[11] Lionel C. Briand, C. Bunse, and J.W. Daly. 2001. A controlled experiment for evaluating quality guidelines on the maintainability of object-oriented designs. *IEEE Transactions on Software Engineering* 27, 6 (jun 2001), 513–530. https://doi.org/10.1109/32.926174

[12] Lionel C. Briand and Jürgen Wüst. 2002. Empirical Studies of Quality Models in Object-Oriented Systems. *Advances in Computers* 56 (2002), 97–166. https://doi.org/10.1016/S0065-2458(02)80005-5

[13] Capgemini. 2012. *Business Cloud: The State of Play Shifts Rapidly.* Technical Report. 24 pages. http://www.in.capgemini.com/business-cloud-the-state-of-play-shifts-rapidly

[14] Jeffrey C. Carver, Letizia Jaccheri, Sandro Morasca, and Forrest Shull. 2010. A checklist for integrating student empirical studies with research and teaching goals. *Empirical Software Engineering* 15, 1 (feb 2010), 35–59. https://doi.org/10.1007/s10664-009-9109-9

[15] Jorge Cham and Daniel Whiteson. 2017. *We Have No Idea: A Guide to the Unknown Universe.* Riverhead Books, London. 368 pages.

[16] G. Costagliola, F. Ferrucci, G. Tortora, and G. Vitiello. 2005. Class point: an approach for the size estimation of object-oriented systems. *IEEE Transactions on Software Engineering* 31, 1 (jan 2005), 52–74. https://doi.org/10.1109/TSE.2005.5

[17] Shirley Cruz, Fabio Q.B. da Silva, and Luiz Fernando Capretz. 2015. Forty years of research on personality in software engineering: A mapping study. *Computers in Human Behavior* 46 (may 2015), 94–113. https://doi.org/10.1016/j.chb.2014.12.008

[18] Tore Dybå, Vigdis By Kampenes, and Dag I.K. Sjøberg. 2006. A systematic review of statistical power in software engineering experiments. *Information and Software Technology* 48, 8 (aug 2006), 745–755. https://doi.org/10.1016/j.infsof.2005.08.009

[19] Joe Emison. 2018. *2018 State of the Cloud.* Technical Report. Interop ITX. 38 pages. http://reg.interop.com/stateofcloud{_}2018?kcode=cync

[20] Janet Feigenspan, Christian Kastner, Jorg Liebig, Sven Apel, and Stefan Hanenberg. 2012. Measuring programming experience. In *2012 20th IEEE International Conference on Program Comprehension (ICPC).* IEEE, Passau, 73–82. https://doi.org/10.1109/ICPC.2012.6240511

[21] A Field, J Miles, and Z Field. 2012. *Discovering Statistics Using R.* SAGE Publications, London, UK. 992 pages.

[22] Gordon L. Freeman and Stephen R. Schach. 2005. The task-dependent nature of the maintenance of object-oriented programs. *Journal of Systems and Software* 76, 2 (may 2005), 195–206. https://doi.org/10.1016/j.jss.2004.05.010

[23] Matthias Galster and Eva Bucherer. 2008. A Taxonomy for Identifying and Specifying Non-Functional Requirements in Service-Oriented Development. In *2008 IEEE Congress on Services - Part I.* IEEE, Honolulu, HI, 345–352. https://doi.org/10.1109/SERVICES-1.2008.51

[24] Martin Höst, Björn Regnell, and Claes Wohlin. 2000. Using Students as Subjects - A Comparative Study of Students and Professionals in Lead-Time Impact Assessment. *Empirical Software Engineering* 5, 3 (2000), 201–214.

[25] Ronald Jabangwe, Jürgen Börstler, Darja Šmite, and Claes Wohlin. 2014. Empirical evidence on the link between object-oriented measures and external quality attributes: a systematic literature review. *Empirical Software Engineering* 20, 3 (mar 2014), 640–693. https://doi.org/10.1007/s10664-013-9291-7

[26] Magne Jørgensen, Tore Dybå, Knut Liestøl, and Dag I.K. Sjøberg. 2015. Incorrect results in software engineering experiments: How to improve research practices. *Journal of Systems and Software* (mar 2015). https://doi.org/10.1016/j.jss.2015.03.065

[27] Barbara A. Kitchenham. 2010. What's up with software metrics? – A preliminary mapping study. *Journal of Systems and Software* 83, 1 (jan 2010), 37–51. https://doi.org/10.1016/j.jss.2009.06.041

[28] Barbara A. Kitchenham. 2015. Robust statistical methods. In *Proceedings of the 19th International Conference on Evaluation and Assessment in Software Engineering - EASE '15.* ACM Press, New York, New York, USA, 1–6. https://doi.org/10.1145/2745802.2747956

[29] Barbara A. Kitchenham, S.L. Pfleeger, L.M. Pickard, P.W. Jones, D.C. Hoaglin, K. El Emam, and J. Rosenberg. 2002. Preliminary guidelines for empirical research in software engineering. *IEEE Transactions on Software Engineering* 28, 8 (aug 2002), 721–734. https://doi.org/10.1109/TSE.2002.1027796

[30] Andrew J. Ko, Thomas D. LaToza, and Margaret M. Burnett. 2013. A practical guide to controlled experiments of software engineering tools with human participants. *Empirical Software Engineering* 20, 1 (sep 2013), 110–141. https://doi.org/10.1007/s10664-013-9279-3

[31] Ang Li, Xiaowei Yang, Srikanth Kandula, and Ming Zhang. 2010. CloudCmp: comparing public cloud providers. In *Proceedings of the 10th annual conference on Internet measurement - IMC '10.* ACM Press, Melbourne, Australia, 1–14. https://doi.org/10.1145/1879141.1879143

[32] Zheng Li, Liam O'Brien, He Zhang, and Rainbow Cai. 2012. On a Catalogue of Metrics for Evaluating Commercial Cloud Services. In *2012 ACM/IEEE 13th International Conference on Grid Computing.* IEEE, Beijing, 164–173. https://doi.org/10.1109/Grid.2012.15

[33] Joa Sang Lim, Seung Ryul Jeong, and Stephen R. Schach. 2005. An empirical investigation of the impact of the object-oriented paradigm on the maintainability of real-world mission-critical software. *Journal of Systems and Software* 77, 2 (aug 2005), 131–138. https://doi.org/10.1016/j.jss.2004.11.004

[34] Don MacVittie. 2014. *App Dev Priorities Survey.* Technical Report. InformationWeek. 32 pages. https://www.informationweek.com/whitepaper/soa/data-centers/2015-app-dev-priorities-survey/347333

[35] Peter Mell and Timoty Grance. 2011. *NIST Definition of Cloud Computing.* Technical Report. National Institute of Standards and Technology (Special Publication 800-145), Gaithersburg. 7 pages. http://www.nist.gov/customcf/get{_}pdf.cfm?pub{_}id=909616

[36] Microsoft. 2017. *Cloud Service Map.* Technical Report. Microsoft. 37 pages. https://azure.microsoft.com/pt-br/blog/cloud-service-map-for-aws-and-azure-available-now/
[37] James Miller. 2005. Replicating software engineering experiments: a poisoned chalice or the Holy Grail. *Information and Software Technology* 47, 4 (mar 2005), 233–244. https://doi.org/10.1016/j.infsof.2004.08.005
[38] James Miller, John Daly, Murray Wood, Marc Roper, and Andrew Brooks. 1997. Statistical power and its subcomponents âĂŤ missing and misunderstood concepts in empirical software engineering research. *Information and Software Technology* 39, 4 (apr 1997), 285–295. https://doi.org/10.1016/S0950-5849(96)01139-1
[39] Sofia Ouhbi, Ali Idri, Jose Luis Fernandez Aleman, and Ambrosio Toval. 2014. Evaluating Software Product Quality: A Systematic Mapping Study. In *2014 Joint Conference of the International Workshop on Software Measurement and the International Conference on Software Process and Product Measurement*. IEEE, Rotterdam, 141–151. https://doi.org/10.1109/IWSM.Mensura.2014.30
[40] Outsystems. 2017. *The State of Application Development 2017 Research Report.* Technical Report. outsystems, Atlanta. 21 pages. https://www.outsystems.com/1/state-app-development-trends/
[41] Mikhail Perepletchikov and Caspar Ryan. 2011. A Controlled Experiment for Evaluating the Impact of Coupling on the Maintainability of Service-Oriented Software. *IEEE Transactions on Software Engineering* 37, 4 (jul 2011), 449–465. https://doi.org/10.1109/TSE.2010.61
[42] Reginaldo Ré, Rômulo Manciola Meloca, Douglas Nassif Roma, Marcelo Alexandre da Cruz Ismael, and Gabriel Costa Silva. 2018. An empirical study for evaluating the performance of multi-cloud APIs. *Future Generation Computer Systems* 79 (feb 2018), 726–738. https://doi.org/10.1016/j.future.2017.09.003
[43] Mehwish Riaz, Emilia Mendes, and Ewan Tempero. 2009. A systematic review of software maintainability prediction and metrics. In *2009 3rd International Symposium on Empirical Software Engineering and Measurement, ESEM 2009*. IEEE, Lake Buena Vista, FL, 367–377. https://doi.org/10.1109/ESEM.2009.5314233
[44] RightScale. 2018. *State of the Cloud Report.* Technical Report. RightScale. https://www.rightscale.com/lp/state-of-the-cloud/
[45] Juliana de A.G. Saraiva, Micael S. de França, Sérgio C.B. Soares, Fernando J.C.L. Filho, and Renata M.C.R. de Souza. 2015. Classifying Metrics for Assessing Object-Oriented Software Maintainability: A Family of Metrics' Catalogs. *Journal of Systems and Software* 103 (jan 2015), 85–101. https://doi.org/10.1016/j.jss.2015.01.014
[46] W R Shadish, T D Cook, and D T Campbell. 2001. *Experimental and Quasi-experimental Designs for Generalized Causal Inference* (2nd ed.). Cengage Learning, Boston. 656 pages.
[47] Forrest Shull and Raimund L. Feldmann. 2008. Building Theories from Multiple Evidence Sources. In *Guide to Advanced Empirical Software Engineering* (1 ed.), Forrest Shull, Janice Singer, and Dag I. K. Sjøberg (Eds.). Springer London, London, Chapter 13, 337–364. https://doi.org/10.1007/978-1-84800-044-5_13
[48] Gabriel Costa Silva, Louis M. Rose, and Radu Calinescu. 2013. Towards a Model-Driven Solution to the Vendor Lock-In Problem in Cloud Computing. In *2013 IEEE 5th International Conference on Cloud Computing Technology and Science*. IEEE, Bristol, UK, 711–716. https://doi.org/10.1109/CloudCom.2013.131
[49] Gabriel Costa Silva, Louis M. Rose, and Radu Calinescu. 2014. Cloud DSL: A Language for Supporting Cloud Portability by Describing Cloud Entities. In *MD2P2 2014 âĂŞ Model-Driven Development Processes and Practices*. ACM/IEEE, Valencia, Spain, 18–27.
[50] Dag I.K. Sjøberg, B. Anda, E. Arisholm, Tore Dybå, Magne Jorgensen, A. Karahasanovic, E.F. Koren, and M. Vokac. 2002. Conducting realistic experiments in software engineering. In *Proceedings International Symposium on Empirical Software Engineering*. IEEE Comput. Soc, Nara, 17–26. https://doi.org/10.1109/ISESE.2002.1166921
[51] Dag I.K. Sjøberg, J.E. Hannay, O. Hansen, Vigdis By Kampenes, A. Karahasanovic, N.-K. Liborg, and A.C. Rekdal. 2005. A survey of controlled experiments in software engineering. *IEEE Transactions on Software Engineering* 31, 9 (sep 2005), 733–753. https://doi.org/10.1109/TSE.2005.97
[52] Ian Sommerville. 2016. *Software Engineering* (10 ed.). Pearson, Essex. 810 pages.
[53] A. Vargha and H. D. Delaney. 2000. A Critique and Improvement of the CL Common Language Effect Size Statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics* 25, 2 (jan 2000), 101–132. https://doi.org/10.3102/10769986025002101
[54] Stefan Wagner. 2013. Quality Models. In *Software Product Quality Control*. Springer Berlin Heidelberg, Berlin, Heidelberg, 29–89. https://doi.org/10.1007/978-3-642-38571-1_2
[55] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering*. Springer Berlin Heidelberg, Berlin, Heidelberg. 249 pages. https://doi.org/10.1007/978-3-642-29044-2